KUBERNATES

Today Task Overview & Importance (Summary)

Today I worked on **setting up and practicing Kubernetes cluster deployment** in both **local and cloud environments**. First, I configured **Minikube on my local machine**, which helped me understand how Kubernetes works in a single-node local setup for learning and testing.

Then I created a **real Kubernetes cluster on Ubuntu** with **1 Master (Control Plane) and 2 Worker Nodes**, which is an industry-standard architecture used in production environments. This gave me hands-on experience in cluster initialization, node joining, troubleshooting worker node issues, and validating cluster health.

After cluster setup, I deployed an **Nginx pod** and exposed it using **NodePort service**, which helped me understand real-time application deployment, service networking, and how traffic is routed inside Kubernetes.

Finally, I explored the **Master and Worker node components**, which is important to understand how Kubernetes manages scheduling, networking, and container execution.

Importance of This Task

This task is important because it builds strong practical knowledge of:

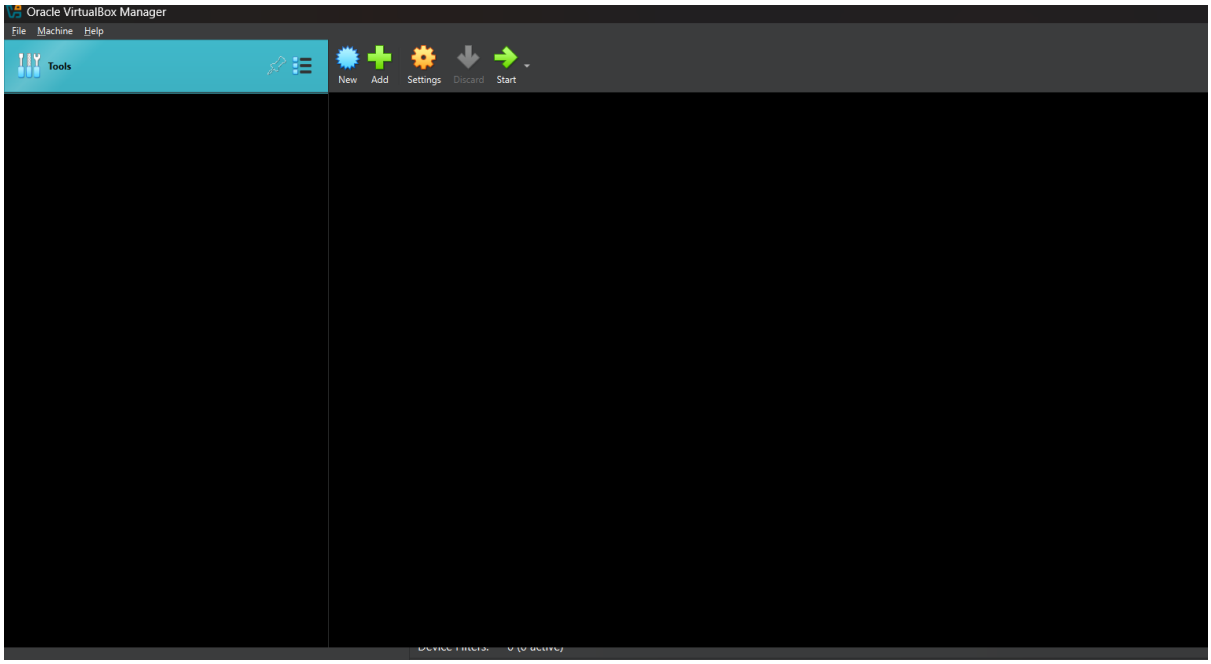
- Kubernetes architecture (Master & Worker responsibilities)
- Cluster setup using kubeadm
- Pod deployment and service exposure
- Troubleshooting real issues (like worker NotReady)
- Real-world DevOps skills used in production deployments

TASK 1: Setup Minikube in Local Machine (Windows)

Step 1: Enable Virtualization

1. Restart PC
2. Enter BIOS (F2 / DEL)
3. Enable **Intel VT-x / AMD-V**

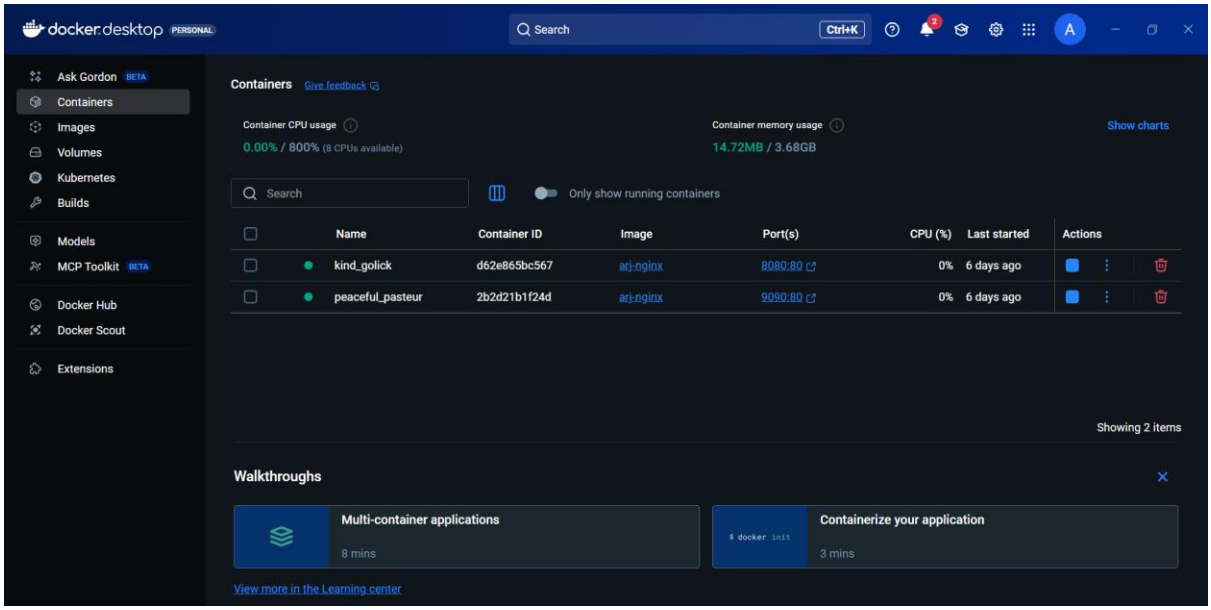
4. Save and Exit



Step 2: Install Docker Desktop

1. Download Docker Desktop from official Docker website
2. Install it
3. Enable these options during installation:
    - ✅ Use WSL2 backend
    - ✅ Add Docker to PATH

Restart PC after installation.



Step 3: Verify Docker Installation

Open PowerShell and run:

docker version

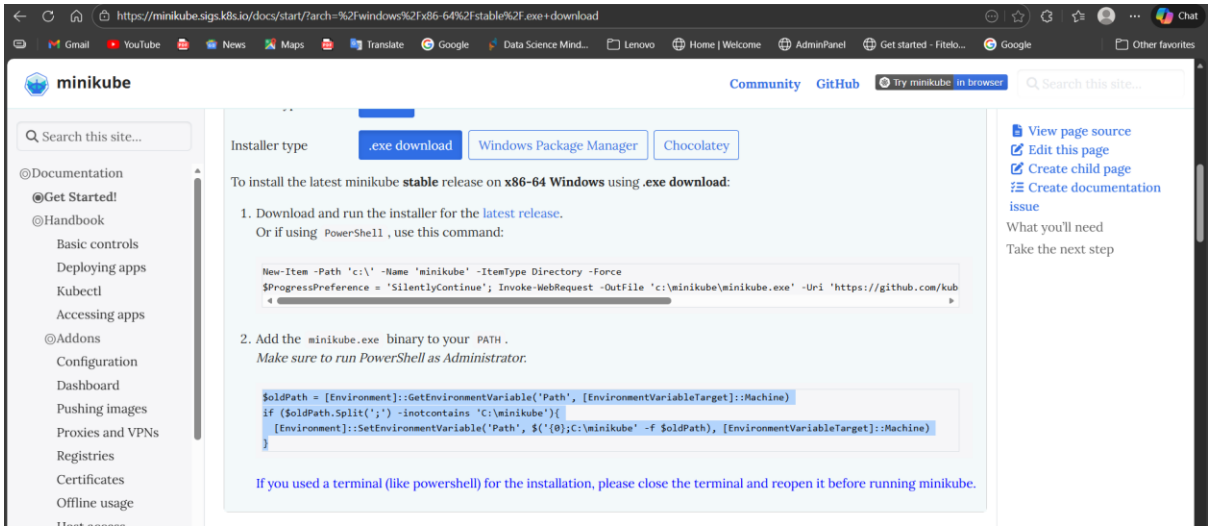
Expected output: Docker Client + Server version.

Step 4: Install kubectl

Download kubectl:

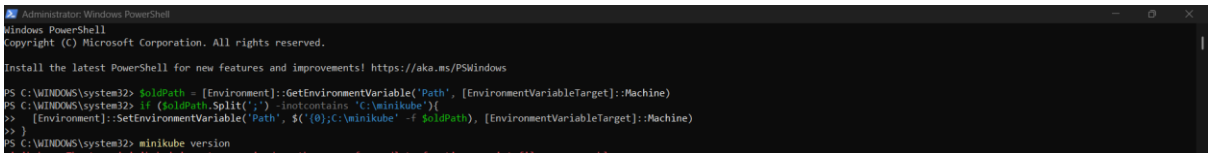
curl.exe -LO "https://dl.k8s.io/release/v1.34.1/bin/windows/amd64/kubectl.exe"

Create folder and move file:



mkdir C:\kubectl

move kubectl.exe C:\kubectl\



Step 5: Add kubectl to PATH

Go to:

Environment Variables → System Variables → Path → Edit → New

Add:

C:\kubectl

Close and reopen PowerShell.

Verify:

kubectl version --client

Step 6: Install Minikube

Create folder:

mkdir C:\minikube

Download minikube:

curl.exe -LO https://storage.googleapis.com/minikube/releases/latest/minikube-windows-amd64.exe

move minikube-windows-amd64.exe C:\minikube\minikube.exe

Step 7: Add Minikube to PATH

Go to:

Environment Variables → System Variables → Path → Edit → New

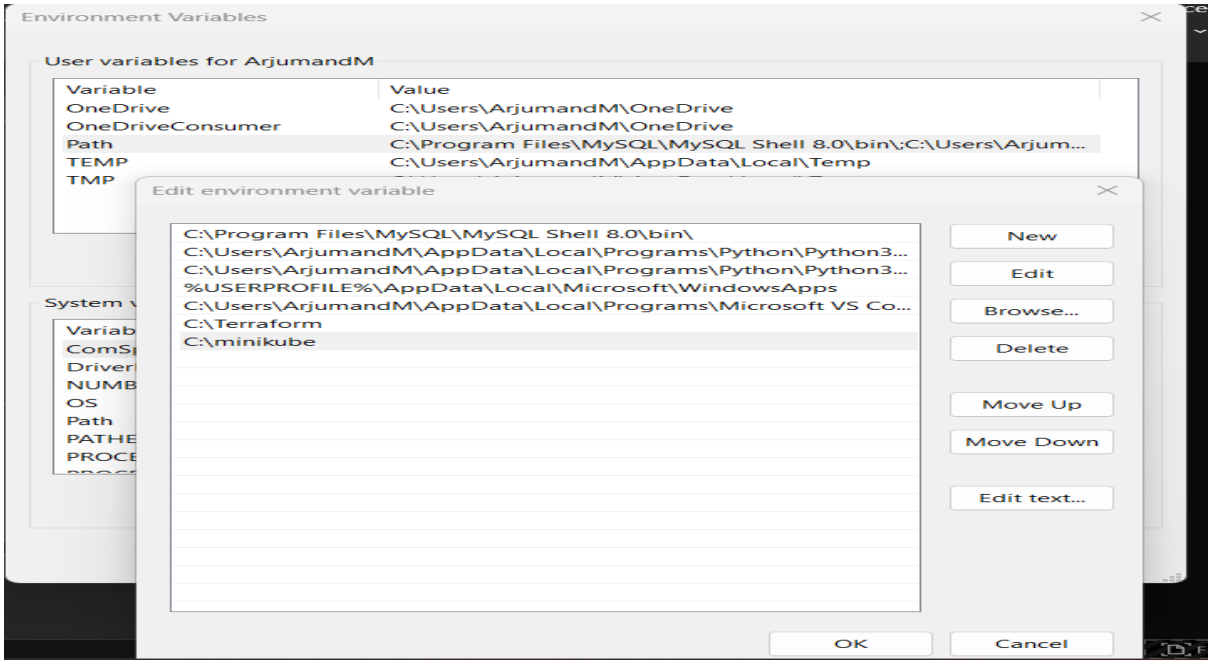
Add:

C:\minikube

Restart PowerShell.

Verify:

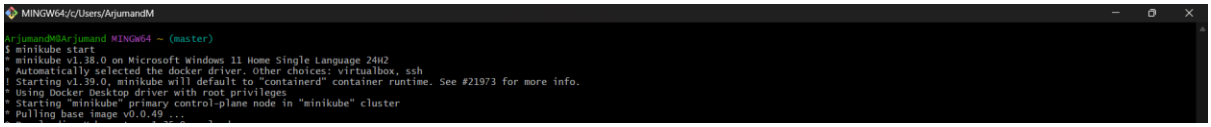
minikube version



Step 8: Start Minikube Cluster

Run:

minikube start --driver=docker



Step 9: Check Minikube Status

Run:

minikube status

Step 10: Check Kubernetes Nodes

Run:

kubectl get nodes

Expected output:

minikube Ready

✅ Minikube setup completed.

☑ TASK 2: Setup Kubernetes Master and Two Worker Nodes (Ubuntu)

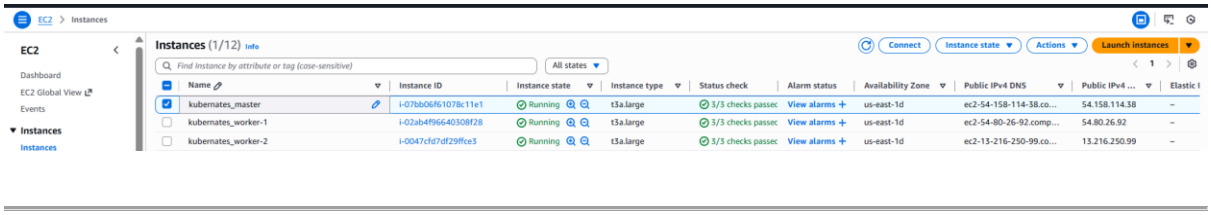
🔥 **Requirement**

You need **3 Ubuntu Machines (VMs or servers)**:

Node Type	Hostname	Example IP
Master	master	192.168.1.10

Node Type	Hostname	Example IP
Worker1	worker1	192.168.1.11
Worker2	worker2	192.168.1.12

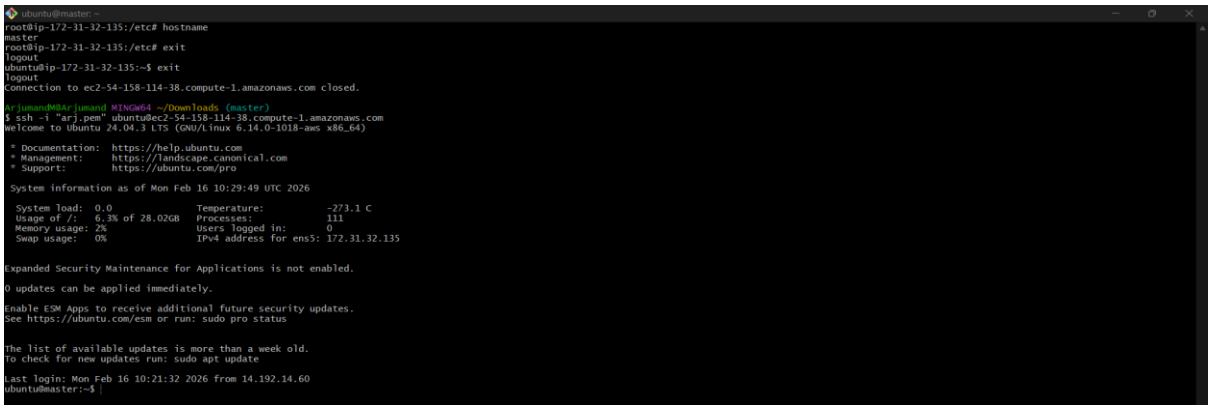
All machines must ping each other.



- **Step 1: Set Hostnames (All Nodes)**

On Master:

sudo hostnamectl set-hostname master



On Worker1:

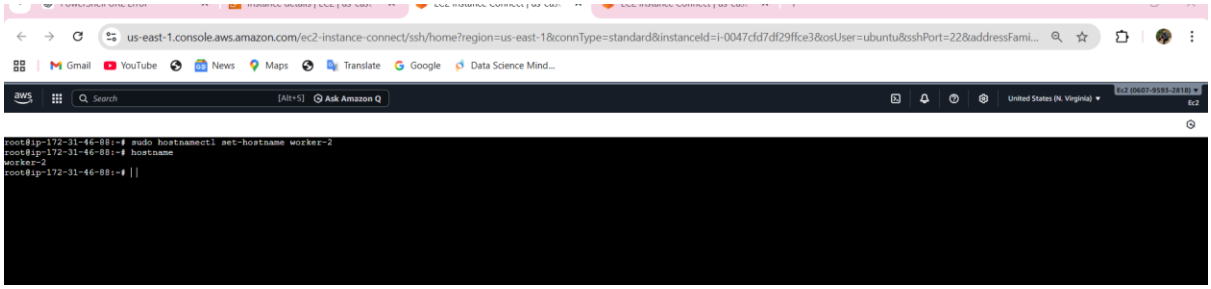
sudo hostnamectl set-hostname worker1

On Worker2:

sudo hostnamectl set-hostname worker2

Restart all nodes:

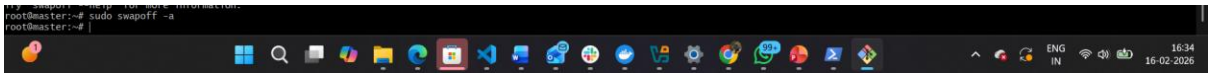
sudo reboot

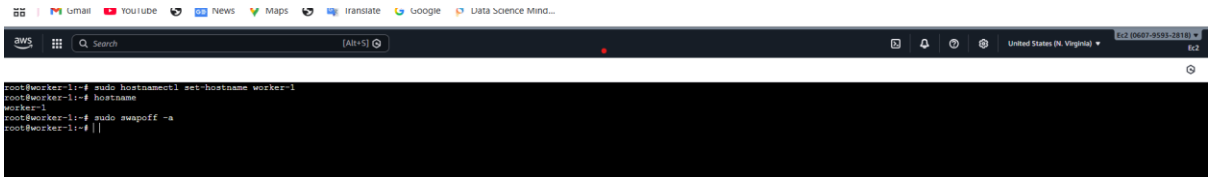


Do The below from steps 1 to 6 in both Master & Worker nodes:-

Step 1: Disable swap:

sudo swapoff -a





Step 3: Update the /etc/hosts File for Hostname Resolution

sudo vi /etc/hosts

Add private ip's of nodes

Example:-

Step 4: Set up the IPV4 bridge on all nodes

cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf

overlay

br_netfilter

EOF

sudo modprobe overlay

sudo modprobe br_netfilter

sysctl params required by setup, params persist across reboots

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf

net.bridge.bridge-nf-call-iptables = 1

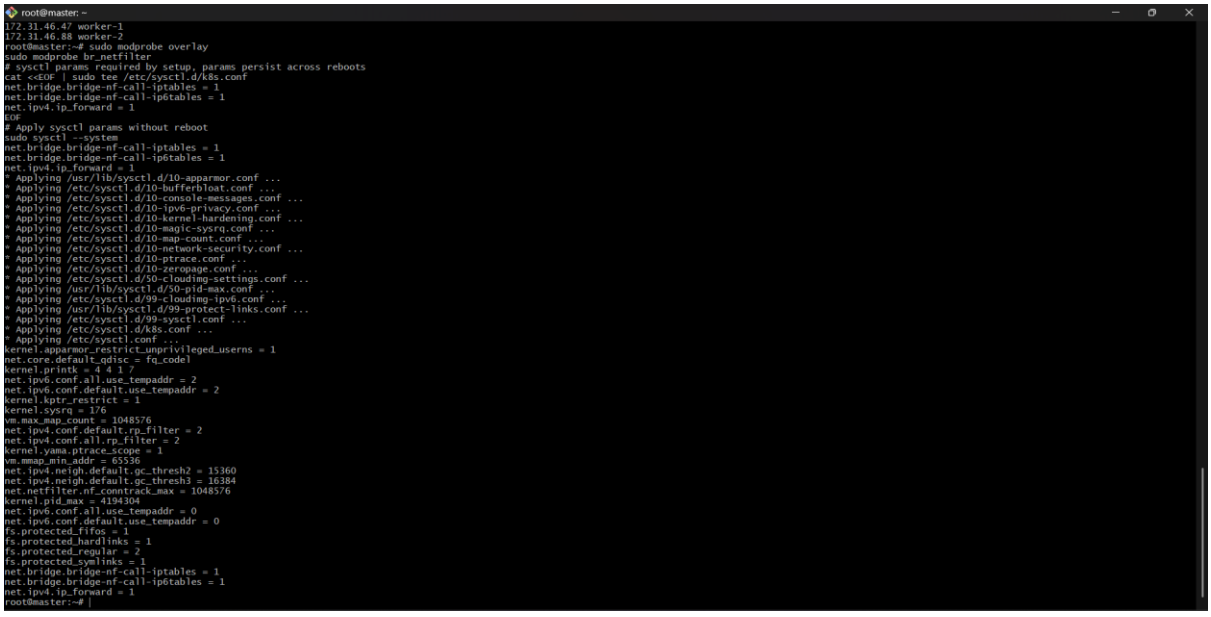
net.bridge.bridge-nf-call-ip6tables = 1

net.ipv4.ip_forward = 1

EOF

Apply sysctl params without reboot

sudo sysctl –system



→worker-1

→Worker-2

Step 5: Install kubelet, kubeadm, and kubectl on each node

sudo apt-get update

sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee

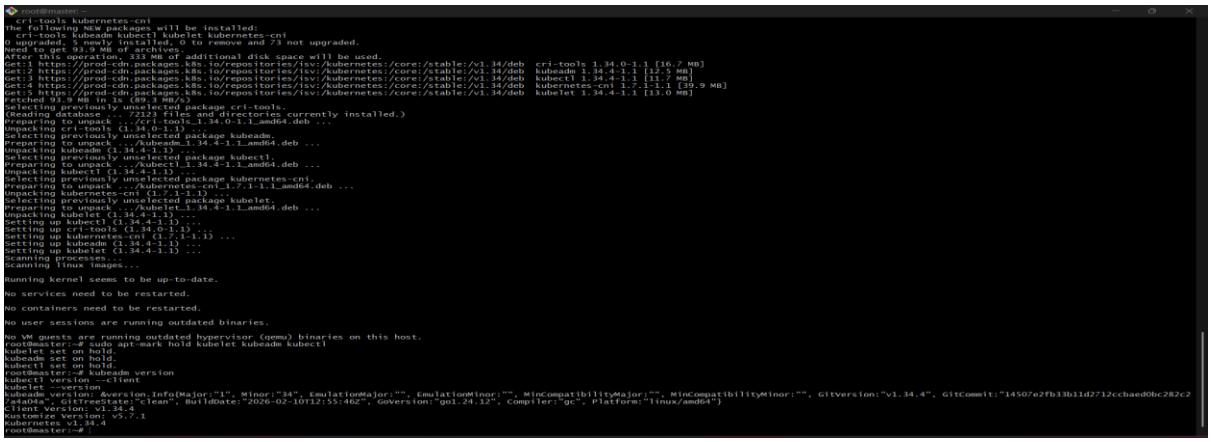
/etc/apt/sources.list.d/kubernetes.list

sudo apt-get update

sudo apt-get install -y kubelet kubeadm kubectl

sudo apt-mark hold kubelet kubeadm kubectl

→Master–



→Worker-1

→Worker-2

Step 6: Install Docker

sudo apt install docker.io

sudo mkdir /etc/containerd

sudo sh -c "containerd config default > /etc/containerd/config.toml"

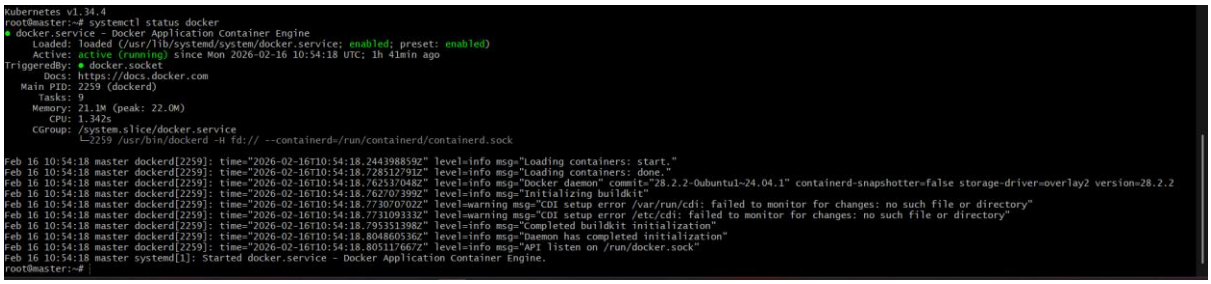
sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml

sudo systemctl restart containerd.service

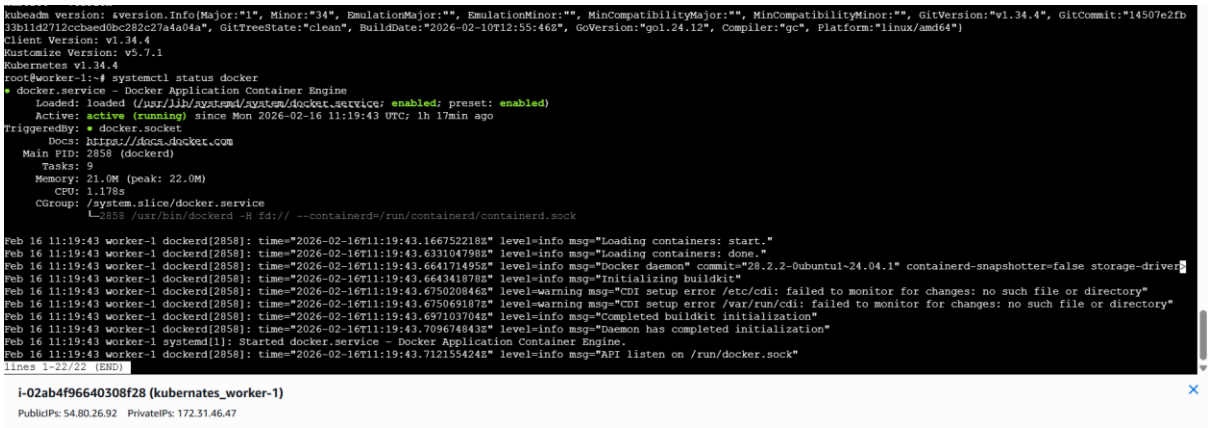
sudo systemctl restart kubelet.service

sudo systemctl enable kubelet.service

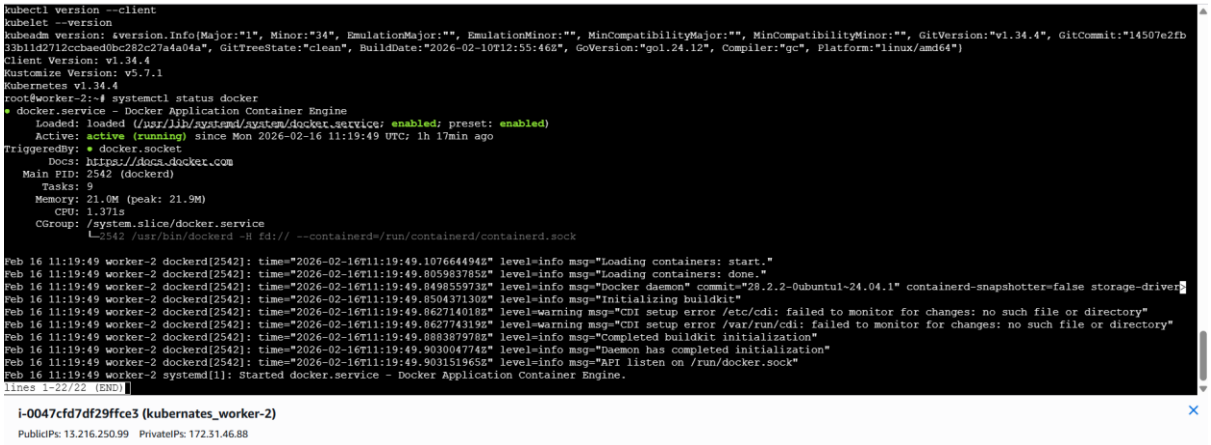
→master



→worker-1



→worker-2



- **Step 3: Execute below commands**

docker --version

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@master:~# docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

systemctl start docker

#systemctl enable docker

systemctl status docker

```
root@master:~# systemctl start docker
root@master:~# systemctl enable docker
root@master:~# systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-02-16 10:54:18 UTC; 54s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 2259 (dockerd)
      Tasks: 9
     Memory: 21.6M (peak: 22.0M)
        CPU: 551ms
     CGroup: /system.slice/docker.service
             └─2259 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

sudo mkdir /etc/containerd

sudo sh -c "containerd config default > /etc/containerd/config.toml"

sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml

sudo systemctl restart containerd.service

sudo systemctl restart kubelet.service

sudo systemctl enable kubelet.service

→master

```
root@master:~# sudo mkdir /etc/containerd
sudo sh -c "containerd config default > /etc/containerd/config.toml"
sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd.service
sudo systemctl restart kubelet.service
sudo systemctl enable kubelet.service
root@master:~# sudo systemctl status containerd --no-pager
● containerd.service - containerd container runtime
     Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-02-16 12:41:09 UTC; 1min 44s ago
       Docs: https://containerd.io
   Main PID: 4969 (containerd)
      Tasks: 8
     Memory: 13.4M (peak: 13.8M)
        CPU: 308ms
     CGroup: /system.slice/containerd.service
             └─4969 /usr/bin/containerd

Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.583911552Z" level=info msg=serving... address=/run/containerd/containerd.sock.ttrpc
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.583998881Z" level=info msg=serving... address=/run/containerd/containerd.sock
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584133814Z" level=info msg="Start subscribing containerd event"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584351746Z" level=info msg="Start recovering state"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584475288Z" level=info msg="Start event monitor"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584508388Z" level=info msg="Start snapshots syncer"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584527928Z" level=info msg="Start cni network conf syncer for default"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584540668Z" level=info msg="Start streaming server"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.585278436Z" level=info msg="containerd successfully booted in 0.057959s"
Feb 16 12:41:09 master systemd[1]: Started containerd.service - containerd container runtime.
root@master:~# sudo systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
     Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; preset: enabled)
    Drop-In: /usr/lib/systemd/system/kubelet.service.d
             └─10-kubeadm.conf
     Active: activating (auto-restart) (Result: exit-code) since Mon 2026-02-16 12:43:02 UTC; 4s ago
       Docs: https://kubernetes.io/docs/
    Process: 5129 ExecStart=/usr/bin/kubelet $KUBELET_KUBECONFIG_ARGS $KUBELET_CONFIG_ARGS $KUBELET_KUBEADM_ARGS $KUBELET_EXTRA_ARGS (code=exited, status=1/FAILURE)
   Main PID: 5129 (code=exited, status=1/FAILURE)
        CPU: 84ms
root@master:~#
```

→Worker-1

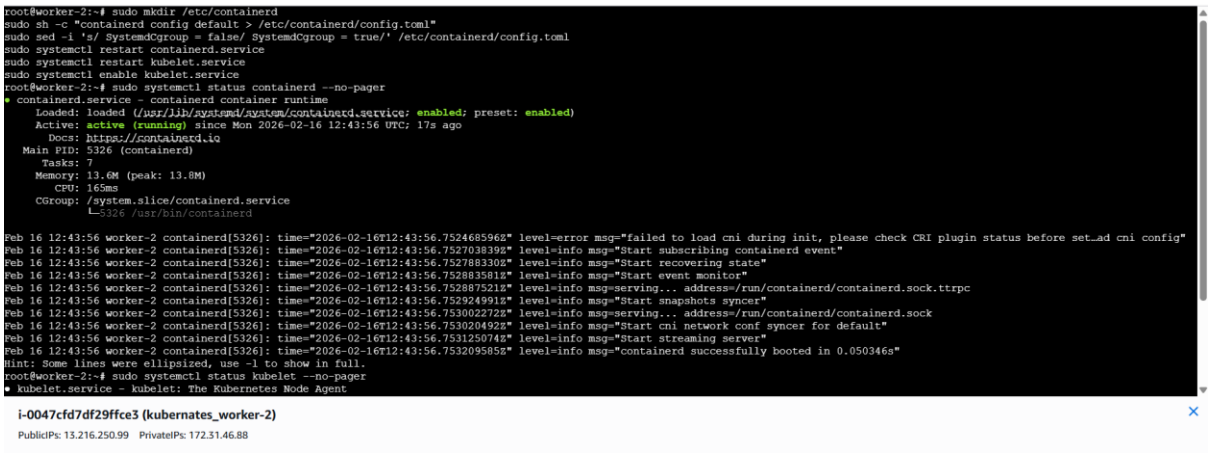
```
     Active: active (running) since Mon 2026-02-16 12:43:50 UTC; 15s ago
       Docs: https://containerd.io
   Main PID: 5272 (containerd)
      Tasks: 7
     Memory: 13.7M (peak: 14.0M)
        CPU: 137ms
     CGroup: /system.slice/containerd.service
             └─5272 /usr/bin/containerd

Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027046645Z" level=error msg="failed to load cni during init, please check CRI plugin status before set…ad cni config"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027248678Z" level=info msg="Start subscribing containerd event"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027313879Z" level=info msg="Start recovering state"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027397121Z" level=info msg="Start event monitor"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027424541Z" level=info msg="Start snapshots syncer"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027442291Z" level=info msg="Start cni network conf syncer for default"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027456402Z" level=info msg="Start streaming server"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027480502Z" level=info msg=serving... address=/run/containerd/containerd.sock.ttrpc
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027555073Z" level=info msg=serving... address=/run/containerd/containerd.sock
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027653855Z" level=info msg="containerd successfully booted in 0.049327s"
Hint: Some lines were ellipsized, use -l to show in full.
root@worker-1:~# sudo systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
     Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; preset: enabled)
    Drop-In: /usr/lib/systemd/system/kubelet.service.d
             └─10-kubeadm.conf
     Active: activating (auto-restart) (Result: exit-code) since Mon 2026-02-16 12:44:11 UTC; 9s ago
       Docs: https://kubernetes.io/docs/
    Process: 5353 ExecStart=/usr/bin/kubelet $KUBELET_KUBECONFIG_ARGS $KUBELET_CONFIG_ARGS $KUBELET_KUBEADM_ARGS $KUBELET_EXTRA_ARGS (code=exited, status=1/FAILURE)
   Main PID: 5353 (code=exited, status=1/FAILURE)
        CPU: 80ms
root@worker-1:~#
```

i-02ab4f96640308f28 (kubernates_worker-1)

PublicIPs: 54.80.26.92 PrivateIPs: 172.31.46.47

→worker-2



Step 7: Initialize the Kubernetes cluster on the master node

On MASTER NODE:-

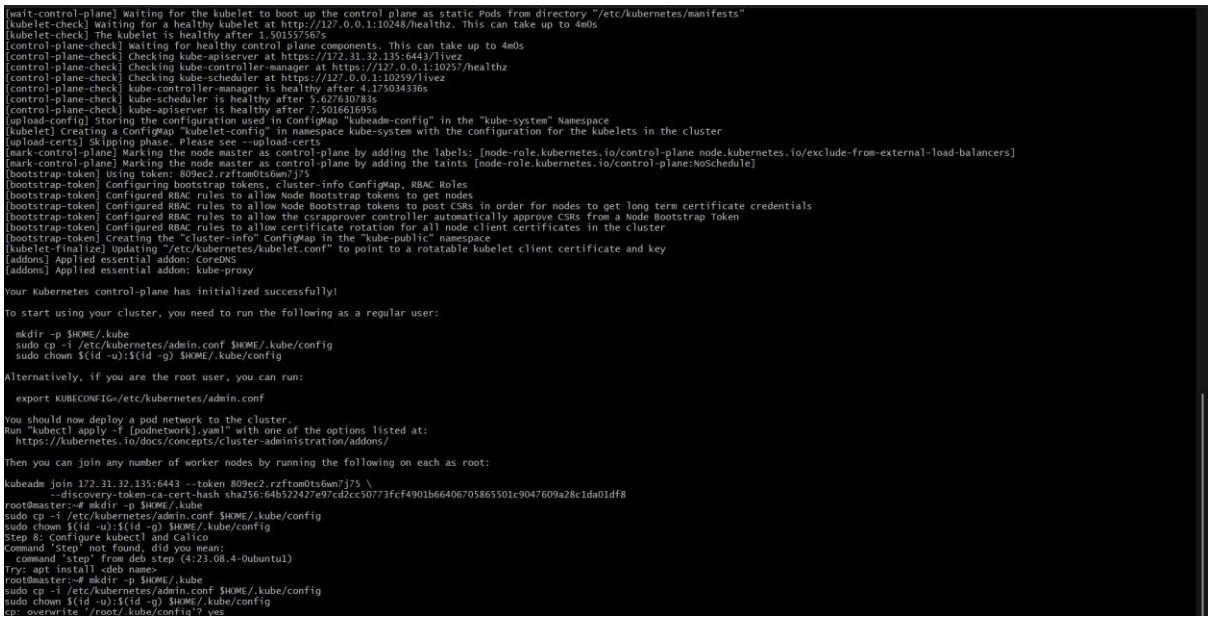
sudo kubeadm config images pull

sudo kubeadm init --pod-network-cidr=10.10.0.0/16

mkdir -p $HOME/.kube

sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config

sudo chown $(id -u):$(id -g) $HOME/.kube/config



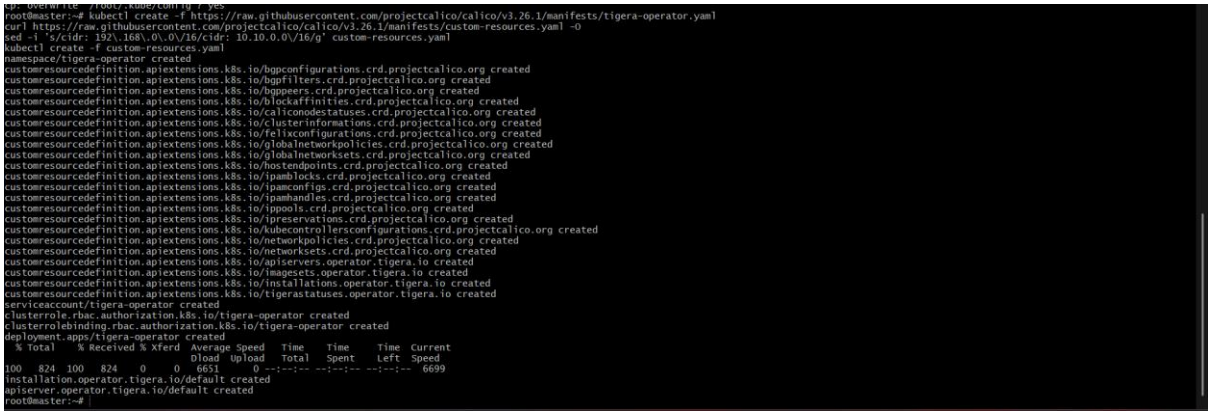
Step 8: Configure kubectl and Calico

kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/tigera-operator.yaml

curl https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/custom-resources.yaml -O

sed -i 's/cidr: 192\.168\.0\.0\/16/cidr: 10.10.0.0\/16/g' custom-resources.yaml

kubectl create -f custom-resources.yaml



Step 9: Add worker nodes to the cluster

sudo kubeadm join <MASTER_NODE_IP>:<API_SERVER_PORT> --token <TOKEN> --discovery-token-ca-cert-hash

<CERTIFICATE_HASH>

Step 10: Verify the cluster and test

kubectl get nodes

TASK 3: Run One Nginx Pod in Kubernetes Cluster

- **Step 1: Create Nginx Pod (MASTER)**

Run:

kubectl run nginx --image=nginx --port=80

- **Step 2: Check Pod Status**

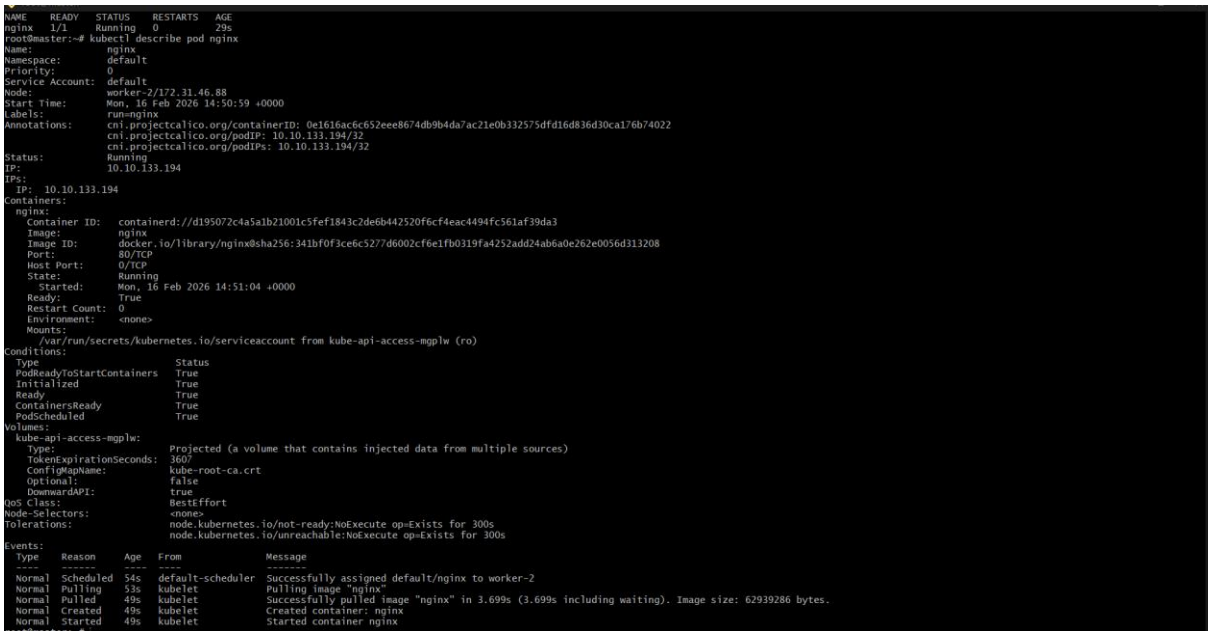
kubectl get pods

Expected:

nginx Running

- **Step 3: Describe Pod (Optional)**

kubectl describe pod nginx



- **Step 4: Expose Nginx Pod as Service**

kubectl expose pod nginx --type=NodePort --port=80

- **Step 5: Check Service**

kubectl get svc

Example output:

nginx NodePort 10.xx.xx.xx <none> 80:30007/TCP

- **Step 6: Access Nginx from Browser**

Open in browser:

http://<worker-ip>:<nodeport>

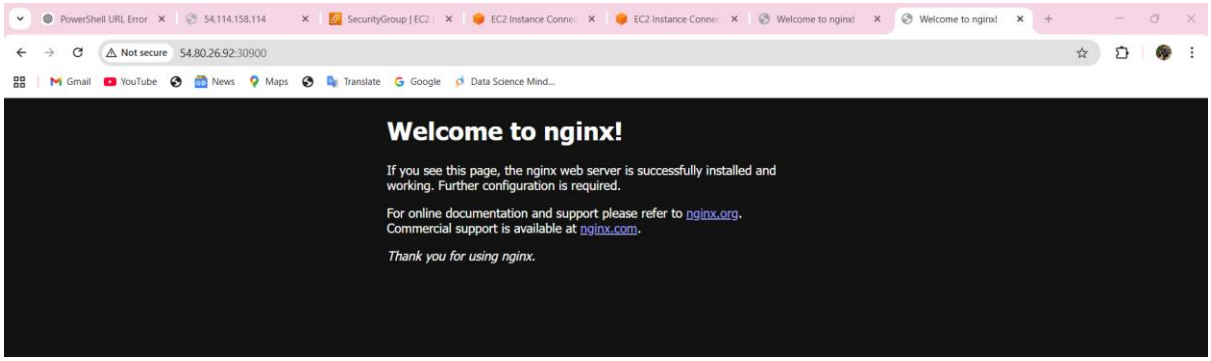
Example:

http://192.168.1.11:30007

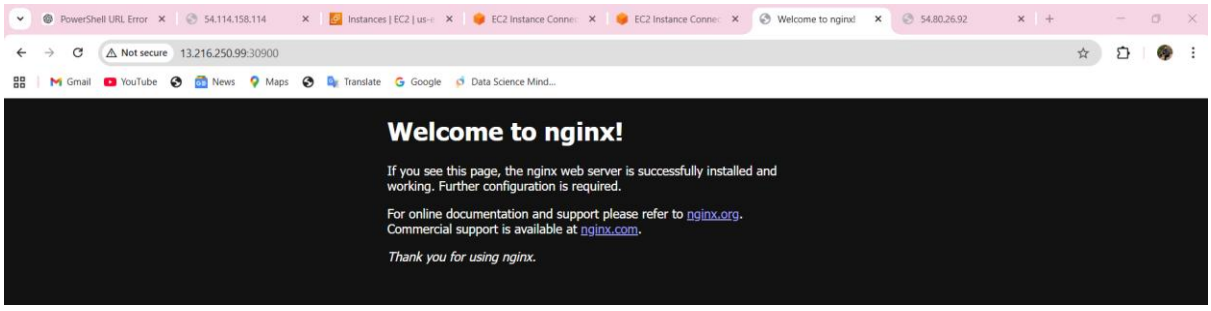
You will see:

✅ **Welcome to nginx!**

→worker-1



→worker-2



✅ **Final Verification Commands**

Check all resources

kubectl get all

```
root@master:~# kubectl get all
NAME                                 READY   STATUS    RESTARTS   AGE
pod/nginx                            1/1     Running   0          2m2s
pod/nginx-deploy-6f47956ff4-5ccl9    1/1     Running   0          115s
pod/nginx-deploy-6f47956ff4-d4c75    1/1     Running   0          115s

NAME                    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/kubernetes      ClusterIP   10.96.0.1       <none>        443/TCP        153m
service/nginx           NodePort    10.106.28.160   <none>        80:30900/TCP   31m
service/nginx-deploy    NodePort    10.104.55.240   <none>        80:32655/TCP   114s

NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx-deploy   2/2     2            2           115s

NAME                                      DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-deploy-6f47956ff4   2         2         2       115s
root@master:~#
```

Check cluster info

kubectl cluster-info

Check pod logs

kubectl logs nginx

```
root@master:~# kubectl cluster-info
Kubernetes control plane is running at https://172.31.32.135:6443
CoreDNS is running at https://172.31.32.135:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
root@master:~# kubectl logs nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/16 15:22:25 [notice] 1#1: using the "epoll" event method
2026/02/16 15:22:25 [notice] 1#1: nginx/1.29.5
2026/02/16 15:22:25 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/16 15:22:25 [notice] 1#1: OS: Linux 6.14.0-1018-aws
2026/02/16 15:22:25 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/16 15:22:25 [notice] 1#1: start worker processes
2026/02/16 15:22:25 [notice] 1#1: start worker process 29
2026/02/16 15:22:25 [notice] 1#1: start worker process 30
172.31.46.47 - - [16/Feb/2026:15:23:33 +0000] "GET / HTTP/1.1" 200 615 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36" "-"
2026/02/16 15:23:33 [error] 29#29: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.31.46.47, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "54.80.26.92:30900", referrer: "http://54.80.26.92:30900/"
172.31.46.47 - - [16/Feb/2026:15:23:33 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://54.80.26.92:30900/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36" "-"
```

✅ Cleanup Commands (Optional)

Delete nginx pod:

kubectl delete pod nginx

Delete nginx service:

kubectl delete svc nginx

```
root@master: ~
root@master:~# kubectl delete pod nginx
pod "nginx" deleted from default namespace
root@master:~# kubectl delete svc nginx
service "nginx" deleted from default namespace
```

✅ Conclusion (For Your Document)

- Minikube was installed locally for single-node Kubernetes practice.
- Kubernetes cluster was built using kubeadm with 1 master and 2 workers.
- Nginx pod was deployed and accessed using NodePort service.

Troubleshooting Worker-1 Node NotReady Issue (Kubernetes Cluster)

🔹 **Problem Statement**

After setting up Kubernetes master and worker nodes, **worker-1 was showing NotReady** when checked from master node.

Command used:

kubectl get nodes -o wide

Output:

worker-1 NotReady

worker-2 Ready

master Ready

Because of this, **NodePort service was not working on worker-1 public IP**.

🔹 **Step 1: Verify Node Status**

From master node, worker-1 was showing:

- Status: **NotReady**

So we confirmed that worker-1 is not healthy.

🔹 **Step 2: Check Detailed Node Issue**

We ran:

kubectl describe node worker-1

In the output, the main error found was:

NodeStatusUnknown

Kubelet stopped posting node status

This indicated that the kubelet service on worker-1 was not running properly or not sending status updates to master.

🔹 **Step 3: Verify Network Connectivity Between Nodes**

We checked if master can reach worker-1:

ping worker-1

Initially, ping was failing (100% packet loss), so we suspected AWS Security Group issue.

Later after updating Security Group inbound rules, ping started working successfully.

This confirmed that **network connectivity was fixed**.

🔹 **Step 4: Check kubelet Service Logs on Worker-1**

Since the error mentioned kubelet issue, we checked logs:

sudo journalctl -u kubelet -n 50 --no-pager

We found the major error:

failed to load kubelet config file

/var/lib/kubelet/config.yaml: no such file or directory

This confirmed that kubelet was crashing continuously because kubelet configuration file was missing.

Also restart counter was high:

restart counter is at 100+

🔹 **Root Cause Identified**

✅ **Root Cause:**

Worker-1 had an incomplete / corrupted kubeadm join process, due to which kubelet config file was missing:

/var/lib/kubelet/config.yaml

🔹 **Step 5: Fix Worker-1 by Resetting kubeadm**

To fix this, we performed a full reset of worker-1 node.

Run on worker-1:

sudo kubeadm reset -f

Then removed old Kubernetes configs:

sudo rm -rf /etc/kubernetes /var/lib/kubelet /var/lib/etcd ~/.kube

Restarted services:

sudo systemctl restart containerd

sudo systemctl restart kubelet

- **Step 6: Rejoin Worker-1 to the Cluster**

We ran the kubeadm join command again on worker-1:

sudo kubeadm join 172.31.32.135:6443 --token 809ec2.rzftom0ts6wn7j75 --discovery-token-ca-cert-hash sha256:64b522427e97cd2cc50773fcf4901b66406705865501c9047609a28c1da01df8

This time the join process completed successfully.

- **Step 7: Verify Worker-1 is Ready**

After rejoining, we checked from master:

kubectl get nodes -o wide

Output:

master Ready

worker-1 Ready

worker-2 Ready

So worker-1 was fixed successfully.

- **Step 8: Validate Kubernetes Cluster Health**

We confirmed cluster is running:

kubectl cluster-info

We also verified pods and system services are running properly:

kubectl get pods -A

- **Step 9: Issue with Nginx Pod Scheduling**

Initially nginx pod was running only on worker-2 because worker-1 was NotReady at that time.

So we created a deployment with 2 replicas to ensure nginx runs on both worker nodes:

kubectl create deployment nginx-deploy --image=nginx

kubectl scale deployment nginx-deploy --replicas=2

Verified pods:

kubectl get pods -o wide

Output showed nginx running on both nodes:

nginx-deploy-xxx worker-1

nginx-deploy-yyy worker-2

✅ **Final Result**

Worker-1 issue resolved successfully.

✓ worker-1 became Ready
✓ kubelet issue fixed
✓ nginx pods running on both worker nodes
✓ NodePort service accessible from both worker public IPs
